

☺ Forms

Another one of the highlights of using Bootstrap is the ability to create forms with ease. As a web developer, styling forms is one of my least favorite tasks. Bootstrap makes it easy with the simple HTML markup and extended classes for different styles of forms.

The basic form structure comes with Bootstrap; there is no need to add any extra helper classes (see [Figure 2-19](#)). If you use the placeholder, keep in mind that it is only supported in newer browsers. In older browsers, no placeholder text will be displayed:

```
<form>
  <fieldset>
    <legend>Legend</legend>
    <label for="name">Labelname</label>
    <input type="text" id="name"
placeholder="Type something...">
    <span class="help-block">Exampleblock-level help
text here.</span>
    <label class="checkbox" for="checkbox">
      <input type="checkbox" id="checkbox">
        Check me out
    </label>
    <button type="submit"
class="btn">Submit</button>
  </fieldset>
</form>
```

Legend

Label name

Example block-level help text here.

☐ Check me out

Figure 2-19. Basic form

Optional Form Layouts

With a few helper classes, you can dynamically update the layout of your form. Bootstrap comes with a few preset styles to choose from.

Search form

Add `.form-search` to the `<form>` tag, and then add `.search-query` to the `<input>` for an input box with rounded corners and an inline submit button (see [Figure 2-20](#)):

```
<form class="form-search">
  <input type="text" class="input-medium search-query">
  <button type="submit" class="btn">Search</button>
</form>
```

 A horizontal search form. It consists of a single-line input field with rounded corners and a light gray border. To the right of the input field is a button with the text "Search". The entire form is enclosed in a thin black rectangular border.
Figure 2-20. Search form

Inline form

To create a form where all of the elements are inline and labels are alongside, add the class `.form-inline` to the `<form>` tag (see [Figure 2-21](#)). To have the label and the input on the same line, use this inline form code:

```
<form class="form-inline">
  <input type="text" class="input-small" placeholder="Email">
  <input type="password" class="input-small" placeholder="Password">

  <label class="checkbox" for="checkbox">
    <input type="checkbox" Remember me
  </label>
  <button type="submit" class="btn">Sign in</button>

</form>
```

 An inline form layout. It contains four elements arranged horizontally: an "Email" input field, a "Password" input field, a checkbox followed by the text "Remember me", and a "Sign in" button. All elements are aligned to the top. The entire form is enclosed in a thin black rectangular border.
Figure 2-21. Inline form

Horizontal form

Bootstrap also comes with a prebaked horizontal form; this one stands apart from the others not only in the amount of markup, but also in the presentation of the form. Traditionally you'd use a table to get a form layout like the one shown in [Figure 2-22](#), but Bootstrap manages to do it without using tables. Even better, if you're using the responsive CSS, the horizontal form will automatically adapt to smaller layouts by stacking the controls vertically.

To create a form that uses the horizontal layout, do the following:

- Add a class of `.form-horizontal` to the parent `<form>` element.
- Wrap labels and controls in a `<div>` with class `.control-group`.
- Add a class of `.control-label` to the labels.
- Wrap any associated controls in a `<div>` with class `.controls` for proper alignment.

The image shows a horizontal form layout. It consists of three rows. The first row has a label 'Email' followed by a text input field with the placeholder 'Email'. The second row has a label 'Password' followed by a password input field with the placeholder 'Password'. The third row has a checkbox labeled 'Remember me' and a 'Sign in' button.

Figure 2-22. Horizontal form

```
<form class="form-horizontal">
  <div class="control-group">
    <label class="control-label" for="inputEmail">Email</label>

    <div class="controls">
      <input type="text" id="inputEmail" placeholder="Email">
    </div>
  </div>
  <div class="control-group">
    <label class="control-label" for="inputPassword">Password</label>
    <div class="controls">
      <input type="password" id="inputPassword" placeholder="Password">
    </div>
  </div>
  <div class="control-group">
    <div class="controls">
      <label class="checkbox" for="checkbox">

        <input type="checkbox"> Remember me
      </label>
      <button type="submit" class="btn">Sign in</button>
    </div>
  </div>
</form>
```

```

    </div>
  </div>
</form>

```

Supported Form Controls

Bootstrap natively supports the most common form controls. Chief among them are input, textarea, checkbox, radio, and select.

Inputs

The most common form text field is the input—this is where users will enter most of the essential form data (see [Figure 2-23](#)). Bootstrap offers support for all native HTML5 input types: text, password, datetime, datetime-local, date, month, time, week, number, email, URL, search, tel, and color:

```
<input type="text" placeholder="Textinput">
```



Figure 2-23. Input

Textarea

The textarea is used when you need multiple lines of input (see [Figure 2-24](#)). You'll find you mainly modify the rows attribute, changing it to the number of rows that you need to support (fewer rows = smaller box, more rows = bigger box):

```
<textarea rows="3"></textarea>
```

```
<textarea rows="3"></textarea>
```



Figure 2-24. Both the :active default and the textarea

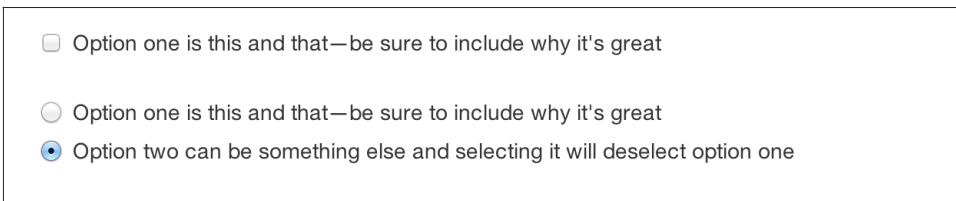
Checkboxes and radio buttons

Checkboxes and radio buttons are great when you want users to choose from a list of preset options (see [Figure 2-25](#)). When building a form, use checkbox if

you want the user to select any number of options from a list. Use radio if you want to limit him to just one selection:

```
<label class="checkbox" for="checkbox">
  <input type="checkbox" value="">
    Option one is this and that—be sure to include why it's great.
</label>

<label>
  <input type="radio" name="optionsRadios" id="optionsRadios1" value="option1"
  checked>
    Option one is this and that—be sure to include why it's great.
</label>
<label>
  <input type="radio" name="optionsRadios" id="optionsRadios2"
  value="option2"> Option two can be something else, and selecting it will
  deselect option one
</label>
```



☐ Option one is this and that—be sure to include why it's great

☐ Option one is this and that—be sure to include why it's great

☒ Option two can be something else and selecting it will deselect option one

Figure 2-25. Checkbox and radio buttons

Selects

A select is used when you want to allow the user to pick from multiple options, but by default it only allows one (see [Figure 2-27](#)). It's best to use `<select>` for list options with which the user is familiar, such as states or numbers. Use `multiple="multiple"` to allow the user to select more than one option. If you only want the user to choose one option, use `type="radio"`:

```
<select>
  <option>1</option>
  <option>2</option>
  <option>3</option>
  <option>4</option>
  <option>5</option>
</select>

<select multiple="multiple">
  <option>1</option>
  <option>2</option>
  <option>3</option>
  <option>4</option>
  <option>5</option>
</select>
```



Figure 2-27. Select

Extended Form Controls

In addition to the basic form controls listed in the previous section, Bootstrap offers a few other form components to complement the standard HTML form elements; for example, it lets you easily prepend and append content to inputs.

Prepended and appended inputs

By adding prepended and appended content to an input field, you can add common elements to the user's input. For example, you can add the dollar symbol, the @ for a Twitter username, or anything else that might be common for your application interface. To add extra content before the user input, wrap the prepended input in a `<div>` with class `.input-prepend`. To append input, use the class `.input-append`.

```
<div class="input-prepend">
  <span class="add-on">@</span>
  <input class="span2" id="prependedInput" type="text" placeholder="Username">
</div>
<div class="input-append">
  <input class="span2" id="appendedInput" type="text">
  <span class="add-on">.00</span>
</div>
```



Figure 2-28. Prepend and append

Form Control Sizing

With the default grid system that is inherent in Bootstrap, you can use the `.span*`

system for sizing form controls. In addition to the span column-sizing method, you can also use a handful of classes that take a relative approach to sizing. If you want the input to act as a block-level element, you can add `.input-block-level` and it will be the full width of the container element, as shown in [Figure 2-32](#):

```
<input class="input-block-level" type="text" placeholder=".input-block-level">
```

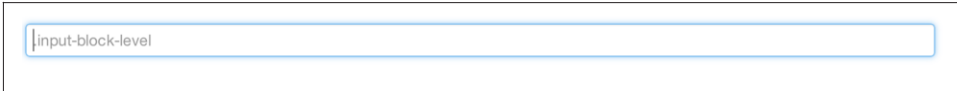


Figure 2-32. Block-level input

Relative input controls

In addition to using `.span*` for input sizing, you can also use a few different class names (see [Figure 2-33](#)):

```
<input class="input-mini" type="text" placeholder=".input-mini">
<input class="form-control input-sm" type="text" placeholder=".input-small">
<input class="form-control input-md" type="text" placeholder=".input-medium">
<input class="form-control input-lg" type="text" placeholder=".input-large">
<input class="form-control input-xlg" type="text" placeholder=".input-xlarge">
<input class="form-control input-xxlg" type="text" placeholder=".input-xxlarge">
```



Figure 2-33. Relative input controls

Grid sizing

You can use any `.span*` from `.span1` to `.span12` for form control sizing (see [Figure 2-34](#)):

```
<input class="span1" type="text" placeholder=".span1">
```

```

<input class="span2" type="text"placeholder=".span2">
<input class="span3" type="text"placeholder=".span3">
<select class="span1">
...
</select>
<select class="span2">
...
</select>
<select class="span3">
...
</select>

```

Figure 2-34. Span-sized inputs

If you want to use multiple inputs on a line, simply use the `.controls-row` modifier class to apply the proper spacing (see [Figure 2-35](#)). It floats the inputs to collapse the white space; sets the correct margins; and, like the `.row` class, clears the float:

```

<div class="controls">
  <input class="span5" type="text"placeholder=".span5">
</div>
<div class="controls controls-row">
  <input class="span4" type="text"placeholder=".span4">
  <input class="span1" type="text"placeholder=".span1">

</div>

```


The figure shows a collection of Bootstrap form controls arranged in a grid-like fashion. It includes several text input fields of different widths, buttons, and spans with class names like .span1, .span2, .span3, .span4, and .span5, demonstrating how they are used to create a control row.

Figure 2-35. Control row

Uneditable text

If you want to present a form control without allowing the user to edit the input, simply add the class `.uneditable-input` (see [Figure 2-36](#)):

```
<span class="input-xlarge uneditable-input">Some value here</span>
```

The figure shows a single Bootstrap form control, a text input field, that is uneditable. It contains the text "Some value here" and has a light gray background, indicating it is not meant to be edited by the user.

Figure 2-36. Uneditable input

Form actions

When you place the form actions at the bottom of a `.horizontal-form`, the inputs will correctly line up with the floated form controls (see [Figure 2-37](#)):

```
<div class="form-actions">
  <button type="submit" class="btn btn-primary">Save changes</button>
  <button type="button" class="btn">Cancel</button>
</div>
```

The figure shows a Bootstrap form control, a horizontal form, with two buttons at the bottom. The first button is labeled "Save changes" and has a blue background. The second button is labeled "Cancel" and has a light gray background. They are aligned horizontally and positioned at the bottom of the form.

Figure 2-37. Form controls

Help text

Bootstrap form controls can have either block or inline text that flows with the

inputs (see [Figure 2-38](#)):

```
<input type="text"><span class="help-inline">Inline help text</span>
```

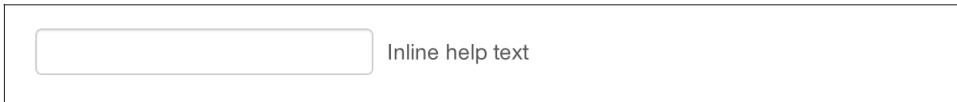


Figure 2-38. Inline help

Form Control States

In addition to the `:focus` state, Bootstrap offers styling for disabled inputs and classes for form validation.

Input focus / Input State

When an input receives `:focus` (i.e., a user clicks into the input or tabs onto it), the outline of the input is removed and a box-shadow is applied. I remember the first time that I saw this on Twitter's site; it blew me away, and I had to dig into the code to see how they did it. In WebKit, this is accomplished in the following manner:

```
input {
  -webkit-box-shadow: inset 0 1px 1px rgba(0, 0, 0, 0.075);
  -webkit-transition: box-shadow linear 0.2s;
}

input:focus {
  -webkit-box-shadow: inset 0 1px 1px rgba(0, 0, 0, 0.075), 0 0 8px
    rgba(82, 168, 236, 0.6);
}
```

The `<input>` has a small inset box-shadow, which gives the appearance that the input sits lower than the page (see [Figure 2-40](#)). When `:focus` is applied, an 8px light blue border appears. The webkit-transition tells the browser to apply the effect in a linear manner over 0.2 seconds:

```
<input class="input-xlarge" id="focusedInput" type="text"
  value="This is focused...">
```



Figure 2-40. Focused input

Nice and subtle; a great effect.

Disabled input / Disabled State

If you need to disable an input, simply adding the disabled attribute will not only disable it; it will also change the styling and the mouse cursor when the cursor hovers over the element (see [Figure 2-41](#)):

```
<input class="input-xlarge" id="disabledInput" type="text"
      placeholder="Disabled input here..." disabled>
```



Figure 2-41. Disabled input

Validation states

Bootstrap includes validation styles for error, warning, info, and success messages (see [Figure 2-42](#)). To use, simply add the appropriate class to the surrounding .control- group:

```
<div class="control-group warning">
  <label class="control-label" for="inputWarning">Input with warning</label>
  <div class="controls">
    <input type="text" id="inputWarning">
    <span class="help-inline">Something may have gone wrong</span>
  </div>
</div>
<div class="control-group error">
  <label class="control-label" for="inputError">Input with error</label>
  <div class="controls">
    <input type="text" id="inputError">

    <span class="help-inline">Please correct the error</span>
  </div>
</div>
<div class="control-group success">
  <label class="control-label" for="inputSuccess">Input with success</label>
  <div class="controls">
    <input type="text" id="inputSuccess">
    <span class="help-inline">Woohoo!</span>
  </div>
</div>
```

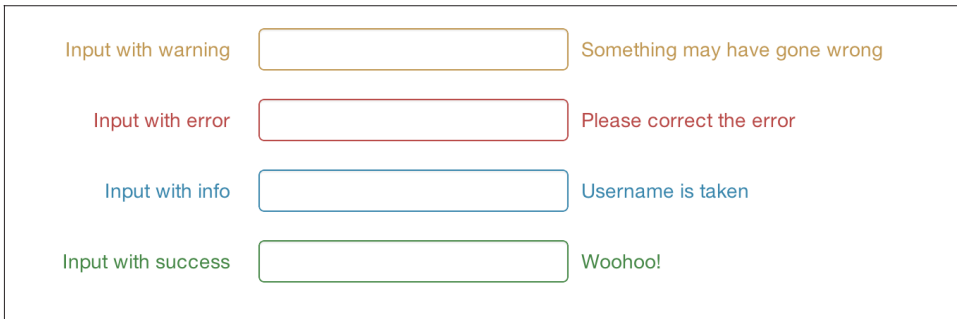


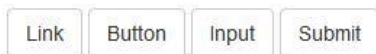
Figure 2-42. Validation states

😊 Buttons

One of the features of Bootstrap is the way that buttons are styled. A grid layout and many of the other features fade into the background, but buttons, forms, and other unifying elements are a key part of Bootstrap.

Now, buttons and links can all look alike with Bootstrap. Anything that is given a class of `.btn` will inherit the default look of a gray button with rounded corners. However, you can add color to the buttons by adding extra classes (see [Table 2-3](#)).

Use the button classes on an `<a>`, `<button>`, or `<input>` element.



```
<a class="btn btn-default" href="#" role="button">Link</a>
```

```
<button class="btn btn-default" type="submit">Button</button>
```

```
<input class="btn btn-default" type="button" value="Input">
```

```
<input class="btn btn-default" type="submit" value="Submit">
```

Context-specific usage

While button classes can be used on `<a>` and `<button>` elements, only `<button>` elements are supported within our `nav` and `navbar` components.

Links acting as buttons

If the `<a>` elements are used to act as buttons – triggering in-page functionality, rather than navigating to another document or section within the current page – they should also be given an appropriate `role="button"`.

Cross-browser rendering

As a best practice, **we highly recommend using the `<button>` element whenever possible** to ensure matching cross-browser rendering.

OPTIONS

Use any of the available button classes to quickly create a styled button.

Table 2-3. Button color examples

Buttons	Class	Description
	<code>btn</code>	Standard gray button with gradient
	<code>btn btn-primary</code> a set of buttons (blue)	Provides extra visual weight and identifies the primary action in a set of buttons (blue)
	<code>btn btn-info</code>	Used as an alternative to the default styles (light blue)
	<code>btn-success</code>	Indicates a successful or positive action (green)
	<code>btn btn-warning</code>	Indicates caution should be taken with this action (orange)
	<code>btn btn-danger</code>	Indicates a dangerous or potentially negative action (red)
	<code>btn btn-inverse</code>	Alternated dark-gray button, not tied to a semantic action or use
	<code>btn btn-link</code>	De-emphasizes a button by making it look like a link while maintaining button behavior

```
<!-- Standard button -->
```

```
<button type="button" class="btn btn-default">Default</button>
```

```
<!-- Provides extra visual weight and identifies the primary
action in a set of buttons -->
```

```
<button type="button" class="btn btn-
primary">Primary</button>
```

```
<!-- Indicates a successful or positive action -->
```

```
<button type="button" class="btn btn-
success">Success</button>
```

```
<!-- Contextual button for informational alert
messages --> <button type="button" class="btn btn-
info">Info</button>
```

```
<!-- Indicates caution should be taken with this action
--> <button type="button" class="btn btn-
warning">Warning</button>
```

```
<!-- Indicates a dangerous or potentially negative
action --> <button type="button" class="btn btn-
danger">Danger</button>
```

```
<!-- Deemphasize a button by making it look like a
link while maintaining button behavior -->
```

Button Sizes

If you need larger or smaller buttons, simply add `.btn-large`, `.btn-small`, or `.btn-minito` links or buttons (see [Figure 2-43](#)):

```
<p>
  <button class="btnbtn-largebtn-primary" type="button">Largebutton</button>
  <button class="btn btn-large" type="button">Large button</button>
</p>
<p>
  <button class="btn btn-primary" type="button">Default button</button>
  <button class="btn" type="button">Default button</button>
</p>
<p>
  <button class="btnbtn-smallbtn-primary" type="button">Smallbutton</button>

  <button class="btn btn-small" type="button">Small button</button>
</p>
<p>
  <button class="btnbtn-minibtn-primary" type="button">Minibutton</button>
```

```
<button class="btn btn-mini" type="button">Mini button</button>
</p>
```

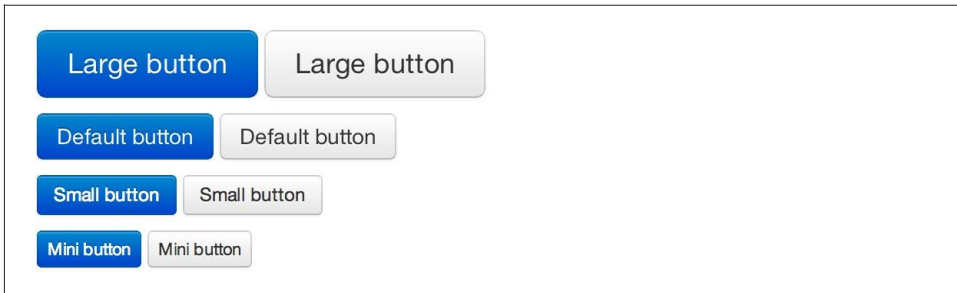


Figure 2-43. Different button sizes

If you want to create buttons that display like a block-level element, simply add the `.btn-block` class (see Figure 2-44). These buttons will display at 100% width:

```
<button class="btn btn-large btn-block btn-primary" type="button">Block-
level button</button>
<button class="btn btn-large btn-block" type="button">Block-level button</button>
```

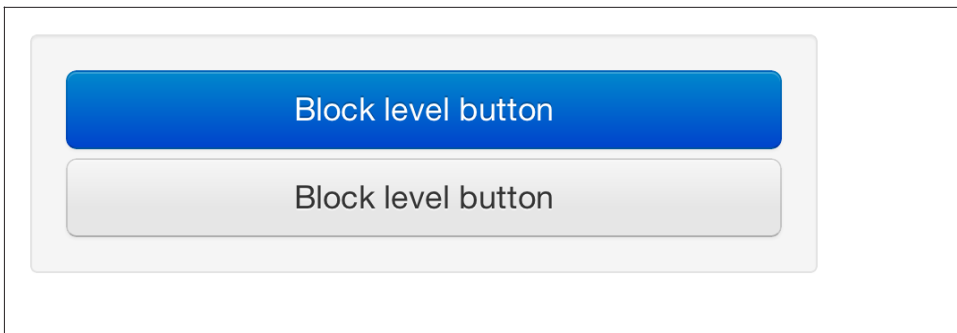


Figure 2-44. Block-level button

Disabled Button Styling

For anchor elements, simply add the class of `.disabled` to the tag and the link will fade in color, and lose the gradient (see Figure 2-45):

```
<a href="#" class="btn btn-large btn-primary disabled">Primary link</a>
<a href="#" class="btn btn-large disabled">Link</a>
```



Figure 2-45. Disabled link

For a button, simply add the disabled attribute to the button (see **Figure 2-46**). This will actually disable the button, so JavaScript is not directly needed:

```
<button type="button" class="btn btn-large btn-primary disabled"
disabled="disabled">Primary button</button>
<button type="button" class="btn btn-large" disabled>Button</button>
```



Figure 2-46. Disabled button

☺ Images

Images have three classes (see **Figure 2-47**) that can be used to apply some simple styles: `.img-rounded` adds `border-radius:6px` to give the image rounded corners, `.img-circle` makes the entire image round by adding `border-radius: 500px`, and `.img-polaroid` adds a bit of padding and a gray border:

```



```



Figure 2-47. Images

RESPONSIVE IMAGES

Images in Bootstrap 3 can be made responsive-friendly via the addition of the .img-responsive class. This applies max-width: 100%;, height: auto; and display: block; to the image so that it scales nicely to the parent element.

To center images which use the .img-responsive class, use .center-block instead of .text-center. [See the helper classes section](#) for more details about .center-block usage.

☺ Icons

Bootstrap bundles 140 icons into one sprite that can be used with buttons, links, navigation, and form fields. The icons are provided by **GLYPHICONS**; see [Figure 2-48](#).

 icon-glass	 icon-music	 icon-search	 icon-envelope
 icon-heart	 icon-star	 icon-star-empty	 icon-user
 icon-film	 icon-th-large	 icon-th	 icon-th-list
 icon-ok	 icon-remove	 icon-zoom-in	 icon-zoom-out
 icon-off	 icon-signal	 icon-cog	 icon-trash
 icon-home	 icon-file	 icon-time	 icon-road
 icon-download-alt	 icon-download	 icon-upload	 icon-inbox
 icon-play-circle	 icon-repeat	 icon-refresh	 icon-list-alt
 icon-lock	 icon-flag	 icon-headphones	 icon-volume-off
 icon-volume-down	 icon-volume-up	 icon-qrcode	 icon-barcode
 icon-tag	 icon-tags	 icon-book	 icon-bookmark
 icon-print	 icon-camera	 icon-font	 icon-bold
 icon-italic	 icon-text-height	 icon-text-width	 icon-align-left
 icon-align-center	 icon-align-right	 icon-align-justify	 icon-list
 icon-indent-left	 icon-indent-right	 icon-facetime-video	 icon-picture
 icon-pencil	 icon-map-marker	 icon-adjust	 icon-tint
 icon-edit	 icon-share	 icon-check	 icon-move
 icon-step-backward	 icon-fast-backward	 icon-backward	 icon-play
 icon-pause	 icon-stop	 icon-forward	 icon-fast-forward
 icon-step-forward	 icon-eject	 icon-chevron-left	 icon-chevron-right
 icon-plus-sign	 icon-minus-sign	 icon-remove-sign	 icon-ok-sign
 icon-question-sign	 icon-info-sign	 icon-screenshot	 icon-remove-circle
 icon-ok-circle	 icon-ban-circle	 icon-arrow-left	 icon-arrow-right
 icon-arrow-up	 icon-arrow-down	 icon-share-alt	 icon-resize-full
 icon-resize-small	 icon-plus	 icon-minus	 icon-asterisk
 icon-exclamation-sign	 icon-gift	 icon-leaf	 icon-fire
 icon-eye-open	 icon-eye-close	 icon-warning-sign	 icon-plane
 icon-calendar	 icon-random	 icon-comment	 icon-magnet
 icon-chevron-up	 icon-chevron-down	 icon-retweet	 icon-shopping-cart
 icon-folder-close	 icon-folder-open	 icon-resize-vertical	 icon-resize-horizontal
 icon-hdd	 icon-bullhorn	 icon-bell	 icon-certificate
 icon-thumbs-up	 icon-thumbs-down	 icon-hand-right	 icon-hand-left
 icon-hand-up	 icon-hand-down	 icon-circle-arrow-right	 icon-circle-arrow-left
 icon-circle-arrow-up	 icon-circle-arrow-down	 icon-globe	 icon-wrench
 icon-tasks	 icon-filter	 icon-briefcase	 icon-fullscreen

Figure 2-48. Icons by GLYPHICONS

Usage

To use the icons, simply use an `<i>` tag with the name spaced `.icon-class`. For example, if you want to use the edit icon, you add the `.icon-edit` class to the `<i>` tag:

```
<i class="icon-edit"></i>
```

If you want to use the white icon, simply add the `.icon-white` class to the tag:

```
<i class="icon-edit icon-white"></i>
```

☺ GLYPHICONS:

Users of Bootstrap are fortunate to use the GLYPHICONS for free on Bootstrap projects. The developers have asked that you link back to GLYPHICONS when practical.

GLYPHICONS Halflings are normally not available for free, but an arrangement between Bootstrap and the GLYPHICONS creators have made this possible at no cost to you as developers. As a thank you, we ask you to include an optional link back to GLYPHICONS whenever practical.

```
<i class="glyphicon glyphicon-book">
```

Example: We use icon with glyphicon element like this:

```
<i class="glyphicon glyphicon-align-left">
<i class="glyphicon glyphicon-align-center">
<i class="glyphicon glyphicon-align-right">
<i class="glyphicon glyphicon-align-justify">
<i class="glyphicon glyphicon-home icon-white">
<i class="glyphicon glyphicon-book">
<i class="glyphicon glyphicon-pencil">
<i class="glyphicon glyphicon-bell">
<ul class="nav nav-list">
  <li class="active"><a href="#"><i class="icon-home icon-
white"></i> Home</a></li>
  <li><a href="#"><i class="icon-book"></i> Library</a></li>
  <li><a href="#"><i class="icon-pencil"></i> Applications</a></li>
  <li><a href="#"><i class="i"></i> Misc</a></li>
</ul>
```



☺ Dropdown Menu

Dropdown menus are toggle able, contextual menus for displaying links in a list format. The dropdowns can be used on a variety of different elements, navs, buttons, and more. You can have a single dropdown or extend the dropdown into another submenu. You can see a basic dropdown menu in **Figure 3-1**.

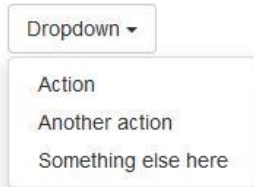


Figure 3-1. Basic dropdown menu

The following code creates a basic dropdown menu:

```
<div class="dropdown">
  <button class="btn btn-default dropdown-toggle" type="button"
id="dropdownMenu1" data-toggle="dropdown" aria-haspopup="true" aria-
expanded="true">
    Dropdown

    <span class="caret"></span> </button>

    <ul class="dropdown-menu" role="menu" aria-
labelledby="dropdownMenu">
      <li><a tabindex="-1" href="#">Action</a></li>
      <li><a tabindex="-1" href="#">Another action</a></li>
      <li><a tabindex="-1" href="#">Something else here</a></li>
    </ul>
```

Options

Right-align

Add `.pull-right` to a `.dropdown-menu` to right-align the dropdown menu to the parent object:

```
<ul class="dropdown-menu pull-right" role="menu" aria-labelledby="dLabel">
  ...
</ul>
```

Submenu

If you would like to add a second layer of dropdowns (see **Figure 3-2**), simply

add .dropdown-submenu to any in an existing dropdown menu for automatic styling:

```
<ul class="dropdown-menu" role="menu" aria-labelledby="dLabel">
  ...
  <li class="dropdown-submenu">
    <a tabindex="-1" href="#">More options</a>
    <ul class="dropdown-menu">
      ...
    </ul>
  </li>
</ul>
```

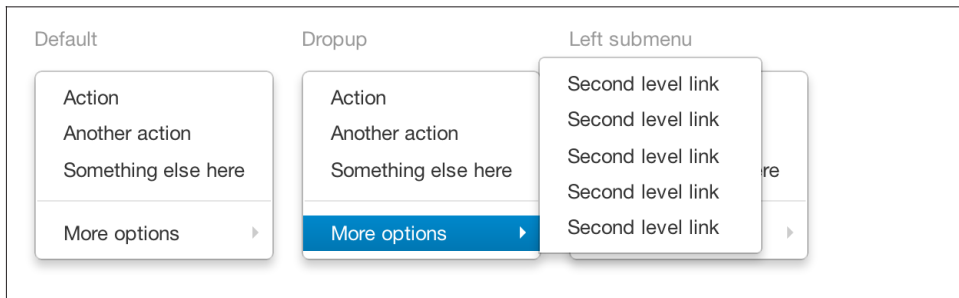


Figure 3-2. Dropdown menu and submenu

HEADERS

Add a header to label sections of actions in any dropdown menu.



```
<ul class="dropdown-menu" aria-labelledby="dropdownMenu3">
  ...
  <li class="dropdown-header">Dropdown header</li>
  ...
</ul>
```

DIVIDER

Add a divider to separate series of links in a dropdown menu.



```
<ul class="dropdown-menu" aria-labelledby="dropdownMenuDivider">
  ...
  <li role="separator" class="divider"></li>
  ...
</ul>
```

DISABLED MENU ITEMS

Add `.disabled` to a `<li>` in the dropdown to disable the link.



```
<ul class="dropdown-menu" aria-labelledby="dropdownMenu4">
<li><a href="#">Regular link</a></li>
<li class="disabled"><a href="#">Disabled link</a></li>
<li><a href="#">Another link</a></li>
</ul>
```

😊 Button Groups

Button groups allow multiple buttons to be stacked together (see **Figure 3-3**). This is useful when you want to place items like alignment buttons together. To create a button group, simply wrap a series of anchors or buttons in a `<div>` that has `.btn-group` as a class:

```
<div class="btn-group">
  <button class="btn">1</button>
  <button class="btn">2</button>
  <button class="btn">3</button>
</div>
```



Figure 3-3. Left, middle, and right button group

By using button groups combined with icons, you can create nice interface elements with minimal markup. If you have multiple button groups (see **Figure 3-**

4) that you want to place on a single line, wrap multiple .btn-groupclasses with .btn-toolbar:

```
<div class="btn-toolbar">
<div class="btn-group">
<a class="btn" href="#"><i class="glyphicon glyphicon-align-left"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-align-center"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-align-right"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-align-justify"></i></a>
</div>
<div class="btn-group">
<a class="btn" href="#"><i class="glyphicon glyphicon-italic"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-bold"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-font"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-text-height"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-text-width"></i></a>
</div>
<div class="btn-group">
<a class="btn" href="#"><i class="glyphicon glyphicon-indent-left"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-indent-right"></i></a>
</div>
<div class="btn-group">
<a class="btn" href="#"><i class="glyphicon glyphicon-envelope"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-music"></i></a>
<a class="btn" href="#"><i class="glyphicon glyphicon-film"></i></a>
</div>
```

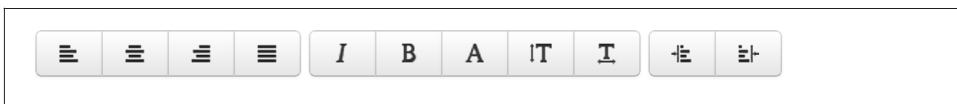


Figure 3-4. Button toolbar

VERTICAL VARIATION

Make a set of buttons appear vertically stacked rather than horizontally. **Split button dropdowns are not supported here.** To stack the buttons vertically (see Figure 3-5), add .btn-group-vertical to the .btn- groupclass:

```
<div class="btn-group btn-group-vertical">
...
</div>
```



Figure 3-5. Vertical button group

SIZING

Instead of applying button sizing classes to every button in a group, just add `.btn-group-*` to each `.btn-group`, including when nesting multiple groups.



```
<div class="btn-group btn-group-lg" role="group" aria-label="...">...</div>
```

```
<div class="btn-group" role="group" aria-label="...">...</div>
```

```
<div class="btn-group btn-group-sm" role="group" aria-label="...">...</div>
```

```
<div class="btn-group btn-group-xs" role="group" aria-label="...">...</div>
```

Button Groups as Radio Buttons and Checkboxes

To have the checkboxes function as radio buttons, where only one option can be selected at a time, or checkboxes, where multiple options can be selected, you simply need to add some extra markup and then Bootstrap's JavaScript will provide the rest.

☺ Buttons with Dropdowns

To add a dropdown to a button (see [Figure 3-6](#)), simply wrap the button and dropdown menu in a `.btn-group`. You can also use `<span class="caret"></span>` to act as an indicator that the button is a dropdown:

Single button dropdowns

```
<div class="btn-group">
  <button class="btn btn-danger">Danger</button>
  <button class="btn btn-danger dropdown-toggle" data-
toggle="dropdown">
    <span class="caret"></span>
  </button>
  <ul class="dropdown-menu">
    <li><a href="#">Action</a></li>
    <li><a href="#">Another action</a></li>
    <li><a href="#">Something else here</a></li>
    <li class="divider"></li>
    <li><a href="#">Separated link</a></li>
  </ul>
</div>
```

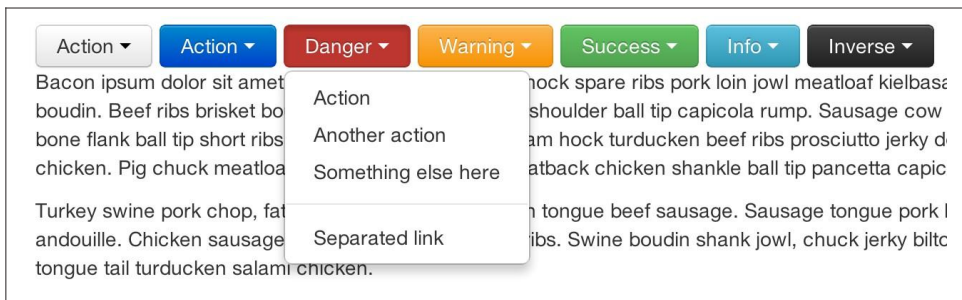


Figure 3-6. Button with a dropdown

You can use the dropdowns with any button size: `.btn-large`, `.btn`, `.btn-small`, or `.btn-mini`. [Figure 3-7](#) shows several examples of different button sizes.

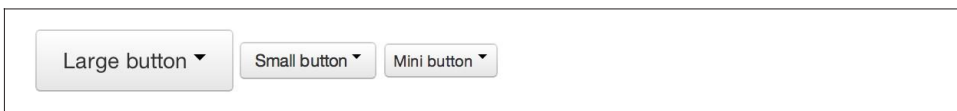


Figure 3-7. Button dropdown sizes

Split Button Dropdowns

Split button dropdowns (see [Figure 3-8](#)) use the same general style as the dropdown button but add a primary action along with the dropdown. Split buttons have the primary action on the left and a toggle on the right that displays the dropdown.

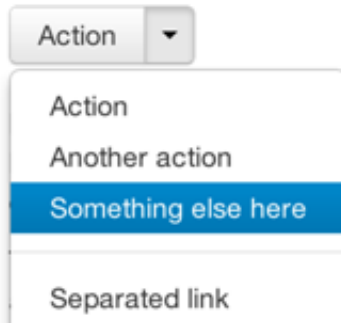


Figure 3-8. Split button dropdown

Here's the code for a split button dropdown:

```
<div class="btn-group">
  <button
    class="btn">Action</button>
  <button class="btn dropdown-toggle" data-toggle="dropdown">
    <span class="caret"></span>
  </button>
  <ul class="dropdown-menu">
    <!-- dropdown menu links -->
  </ul>
</div>
```

Dropup Menus

Menus can also be built to drop up rather than down (see [Figure 3-9](#)). To make this change, simply add `.dropup` to the `.btn-group` container. To have the button pull up from the righthand side, add `.pull-right` to the `.dropdown-menu` (take note: the caret is now pointed up because the menu will be going up instead of down):

```
<div class="btn-group dropup">
  <button
    class="btn">Dropup</button>
  <button class="btn dropdown-toggle" data-toggle="dropdown">
    <span class="caret"></span>
  </button>
  <ul class="dropdown-menu">
    <!-- dropdown menu links -->
  </ul>
</div>
```

```

</ul>
</div>

```



Figure 3-9. Dropdown menu

☺ Navigations or Navigation Elements:

Navs available in Bootstrap have shared markup, starting with the base `.nav` class, as well as shared states. Swap modifier classes to switch between each style.

Bootstrap provides a few different options for styling navigation elements. All of them share the same markup and base class, `.nav`.

Bootstrap also provides a helper class, `.active`. In principle, it generally adds distinction to the current element and sets it apart from the rest of the navigation elements. You can add this class to the home page links or to the links of the page that the user is currently on.

Tabular Navigation / Tabs

To create a tabbed navigation menu (see **Figure 3-10**), start with a basic unordered list with the base class of `.nav` and add `.nav-tabs`:

```

<ul class="nav nav-tabs">
  <li class="active">
    <a href="#">Home</a>
  </li>
  <li><a href="#">Profile</a></li>
  <li><a href="#">Messages</a></li>
</ul>

```



Figure 3-10. Tabbed navigation

Basic Pills Navigation

To turn the tabs into pills (see [Figure 3-11](#)), use `.nav-pills` instead of `.nav-tabs`:

```
<ul class="nav nav-pills">
  <li class="active">
    <a href="#">Home</a>
  </li>
  <li><a href="#">Profile</a></li>
  <li><a href="#">Messages</a></li>
</ul>
```

*Figure 3-11. Tabbed navigation*

Disabled class

For each of the `.nav` classes, if you add the `.disabled` class, it will create a gray link that also disables the `:hoverstate` (see [Figure 3-12](#)):

```
<ul class="nav nav-pills">
  ...
  <li class="disabled"><a href="#">Home</a></li>
  ...
</ul>
```

*Figure 3-12. Disabled navigation*

The link is still clickable unless the `href` is removed with JavaScript or some other method.

Stackable Navigation

Both tabs and pills are horizontal by default. To make them appear vertically stacked, just add the `.nav-stacked` class. See [Figures 3-13](#) and [3-14](#) for examples of vertically stacked tabs and pills.

Here's the code for stacked tabs:

```
<ul class="nav nav-tabs nav-stacked">
  ...
</ul>
```

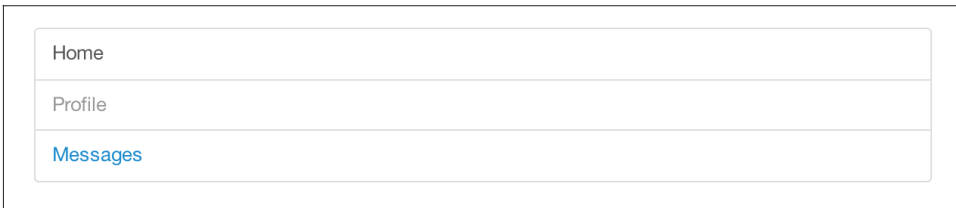


Figure 3-13. Stacked tabs

Dropdowns

Navigation menus share a similar syntax with dropdown menus (see [Figure 3-15](#)). By default, you have a list item that has an anchor working in conjunction with some data- attributes to trigger an unordered list with a `.dropdown-menu` class:

```
<ul class="nav nav-tabs">
  <li class="dropdown">
    <a class="dropdown-toggle"
      data-toggle="dropdown"
      href="#">
      Dropdown
      <b class="caret"></b>
    </a>
    <ul class="dropdown-menu">
      <li><a href="#">Action</a></li>
      <li><a href="#">Another action</a></li>
      <li><a href="#">Something else here</a></li>
      <li class="divider"></li>
      <li><a href="#">Separated link</a></li>
    </ul>
  </li>
</ul>
```

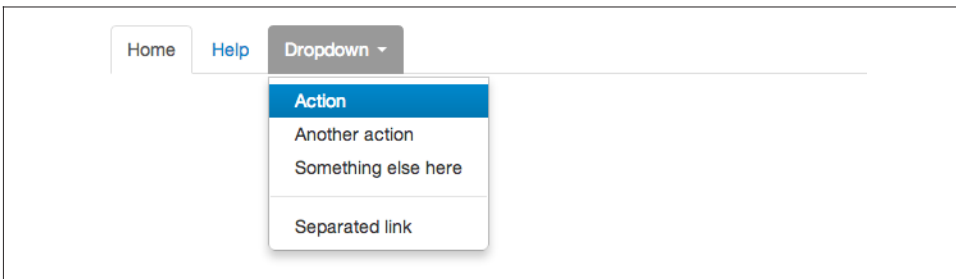
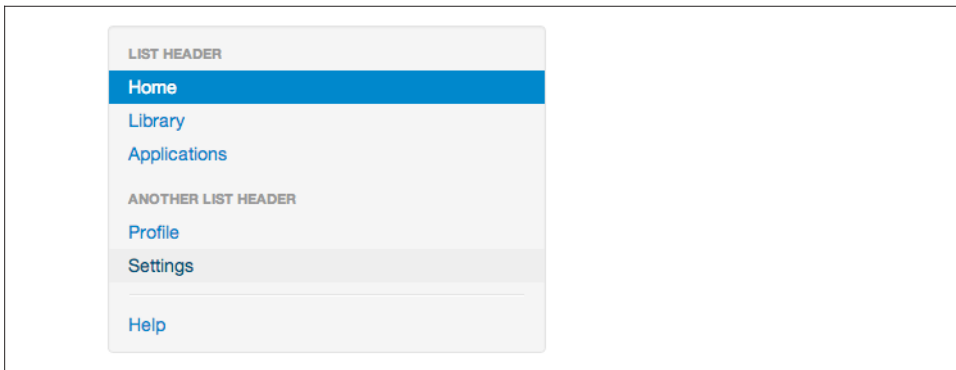


Figure 3-15. Tabbed navigation with a dropdown menu

Navigation Lists

Navigation lists are useful when you need to display a group of navigation links. This type of element is common when building admin interfaces. In the MAKE admin interface, navigation lists are unordered lists with the `.navclass`. To give it its specific styling, we add the `.nav- listclass`:

```
<ul class="nav nav-list">
  <li class="nav-header">List Header</li>
  <li class="active"><a href="/">Home</a></li>
  <li><a href="#">Library</a></li>
  <li><a href="#">Applications</a></li>
  <li class="nav-header">Another List Header</li>
  <li><a href="#">Profile</a></li>
  <li><a href="#">Settings</a></li>
  <li class="divider"></li>
  <li><a href="#">Help</a></li>
</ul>
```



Horizontal divider

To create a divider, much like an `<hr />`, use an empty `<li>` with a class of `.divider`:

```
<ul class="nav-menu">
  ...
  <li class="divider"></li>
  ....
</ul>
```

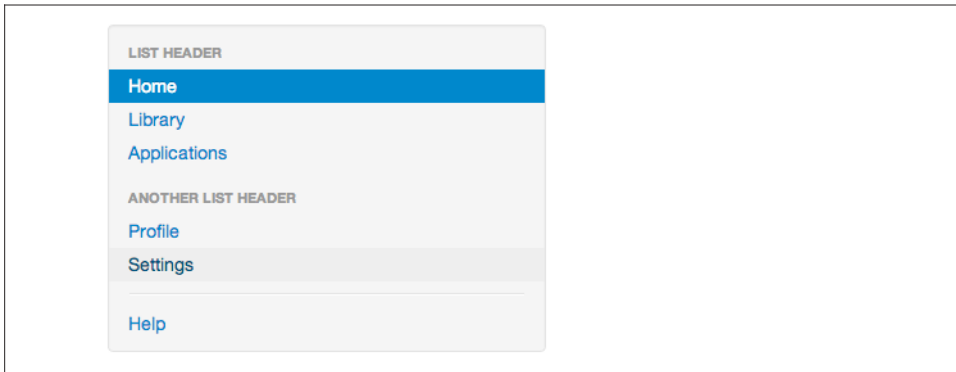


Figure 3-17. Navigation list

Tabbable Navigation

Not only can you create a tabbed navigation, but by using the JavaScript plugin, you can also add interaction by opening different windows of content (see [Figure 3-18](#)). To make navigation tabs, create a `.tab-pane` with a unique ID for every tab, and then wrap them in `.tab-content`:

```
<div class="tabbable">
  <ul class="nav nav-tabs">
    <li class="active"><a href="#tab1" data-toggle="tab">Meats</a></li>
    <li><a href="#tab2" data-toggle="tab">More Meat</a></li>
  </ul>
  <div class="tab-content">
    <div class="tab-pane active" id="tab1">
      <p>Bacon ipsum dolor sit amet jerky flank...</p>
    </div>
    <div class="tab-pane" id="tab2">
      <p>Beef ribs, turducken ham hock...</p>
    </div>
  </div>
</div>
```

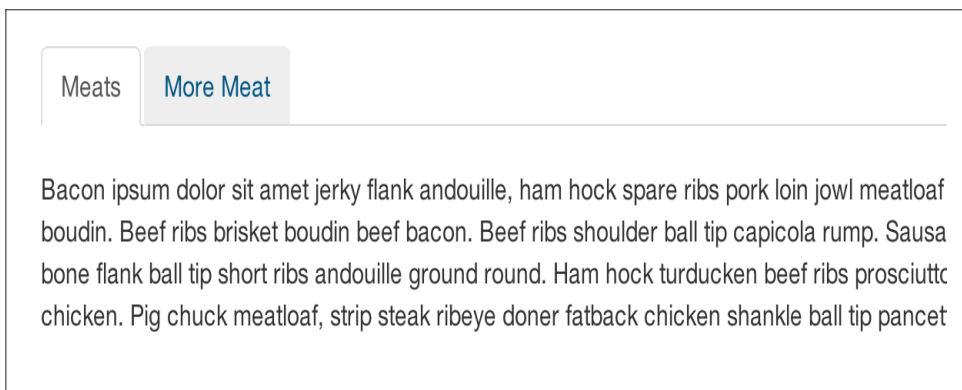
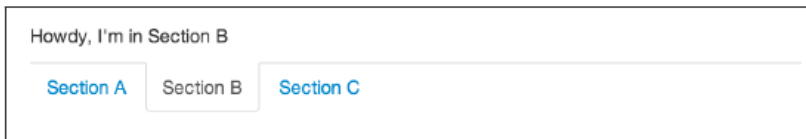


Figure 3-18. Tabbable navigation example

If you want to make the tabs fade when switching, add `.fade` to each `.tab-pane`.

Tab position:

The tabs are fully positionable; you can have them above, below, or on the sides of the content (see [Figure 3-19](#)).

*Figure 3-19. Bottom tabs*

Here's the code for positioning tabs:

```
<div class="tabbable tabs-below">
  <div class="tab-content">
    <div class="tab-pane active" id="tab1">
      <p>I'm in Section A.</p>
    </div>
    <div class="tab-pane" id="tab2">
      <p>I'm in Section B.</p>
    </div>
    <div class="tab-pane" id="tab3">
      <p>I'm in Section C.</p>
    </div>
  </div>
  <ul class="nav nav-tabs">
    <li class="active"><a href="#tab1" data-toggle="tab">Section A</a></li>
    <li><a href="#tab2" data-toggle="tab">Section B</a></li>
    <li><a href="#tab3" data-toggle="tab">Section C</a></li>
  </ul>
</div>
```


😊 Navbar:

The navbar is a nice feature, and is one of the prominent features of Bootstrap sites. Navbars are responsive meta components that serve as navigation headers for your application or site. They begin collapsed (and are toggleable) in mobile views and become horizontal as the available viewport width increases.

At its core, the navbar includes styling for site names and basic navigation. It can later be extended by adding form-specific controls and specialized dropdowns. To be sure that the navbar is constrained to the width of the content of the page, either place it inside of a `.span12` or the `.container` class:

```
<div class="navbar">
  <div class="navbar-inner">
    <a class="brand" href="#">Title</a>
    <ul class="nav">
      <li class="active"><a href="#">Home</a></li>
      <li><a href="#">Link</a></li>
      <li><a href="#">Link</a></li>
    </ul>
  </div>
</div>
```

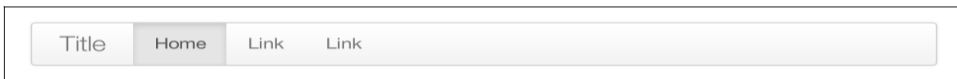


Figure 3-22. Basic navbar

Note the `.brand` class in the code. This will give the text a lighter font-weight and slightly larger size.

```
<a class="brand" href="#">Project name</a>
```

Navbar Links

To add links to the navbar (see [Figure 3-23](#)), simply add an unordered list with a class of `.nav`. If you want to add a divider to your links, you can do that by adding an empty list item with a class of `.divider-vertical`:

```
<ul class="nav">
  <li class="active"><a href="#">Home</a></li>
  <li><a href="#">First Link</a></li>
  <li><a href="#">Second Link</a></li>

  <li class="divider-vertical"></li>
  <li><a href="#">Third Link</a></li>
</ul>
```

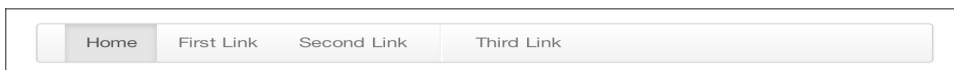


Figure 3-23. Nav link

Forms

Instead of using the default class-based forms from **Chapter 2**, forms that are in the navbar use the `.navbar-form` class. This ensures that the forms margins are properly set and match the nav stylings (see **Figure 3-24**). Of note, `.pull-left` and `.pull-right` helper classes may help move the form into the proper position:

```
<form class="navbar-form pull-left">
  <input type="text" class="span2" id="fname">
  <button type="submit" class="btn">
</form>
```

Figure 3-24. Default navbar form

To add rounded corners (see **Figure 3-25**), as seen in the search inputs of iOS devices, use the `.navbar-search` class instead of the `.navbar-form`:

```
<form class="navbar-search" accept-charset="utf-8">
  <input type="text" class="search-query" placeholder="Search">
</form>
```

Figure 3-25. Navbar search input

Navbar Menu Variations

The Bootstrap navbar can be dynamic in its positioning. By default, it is a block-level element that takes its positioning based on its placement in the HTML. With a few helper classes, you can place it either on the top or bottom of the page, or you can make it scroll statically with the page.

Fixed top navbar

If you want the navbar fixed to the top, add `.navbar-fixed-top` to the `.navbar` class. To prevent the navbar from sitting on top of other content in the body of the page, add at least 40 pixels of padding to the `<body>` tag:

```
<nav class="navbar navbar-default navbar navbar-fixed-top">
  <div class="navbar-header">
    <a class="navbar-brand" href="#">Title</a>
  <ul class="nav navbar-nav">
```

```

        <li class="active"><a href="#">Home</a></li>
        <li><a href="#">Link</a></li>
        <li><a href="#">Link</a></li>
    </ul>
</div>
</nav>

```

Title	Home	Link	Link
-------	------	------	------

Fixed bottom navbar

To affix the navbar to the bottom of the page, simply add the `.fixed-navbar-bottom` class to the navbar. Once again, to prevent overlap, add at least 40 pixels of padding to the `<body>` tag:

```

<div class="navbar navbar-fixed-bottom">
  <div class="navbar-inner">
    <a class="brand" href="#">Title</a>
    <ul class="nav">
      <li class="active"><a href="#">Home</a></li>
      <li><a href="#">Link</a></li>
      <li><a href="#">Link</a></li>
    </ul>
  </div>
</div>

```

Static top navbar

To create a navbar that scrolls with the page, add the `.navbar-static-top` class. This class does not require adding the padding to the `<body>`:

```

<div class="navbar navbar-static-top">
  <div class="navbar-inner">
    <a class="brand" href="#">Title</a>
    <ul class="nav">
      <li class="active"><a href="#">Home</a></li>
      <li><a href="#">Link</a></li>
      <li><a href="#">Link</a></li>
    </ul>
  </div>
</div>

```

Responsive navbar

Like the rest of Bootstrap, the navbar can be totally responsive as shown in **Figure 3-26**. To add the responsive features, the content that you want to be collapsed needs to be wrapped in a `<div>` with `.nav-collapse.collapse` as a class. The collapsing nature is tripped by a button that has a the class of `.btn-navbar` and then features two data-elements. Three `<spans>` with a class of `.icon-bar` create what I like to call the hamburger button. This will toggle the elements that are in the `.nav-collapse<div>`.

Inverted navbar

To create an inverted navbar with a black background and white text as shown in **Figure 3-27**, simply add `.navbar-inverse` to the `.navbar` class:

```
<div class="navbar navbar-inverse">
    ...
</div>
```

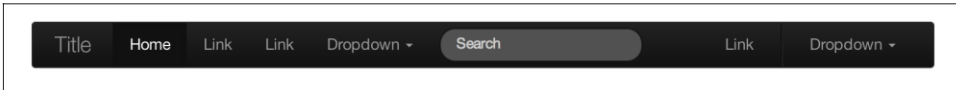


Figure 3-27. Inverted navbar

☺ Breadcrumbs:

Breadcrumbs are a great way to show hierarchy-based information for a site. In the case of blogs, breadcrumbs can show the dates of publishing, categories, or tags. A breadcrumb in Bootstrap is simply an unordered list with a class of `.breadcrumb`. There is also a helper class of `.divider` that mutes the colors and makes the text a little smaller. You can use forward slashes, arrows, or any divided that you choose..

Indicate the current page's location within a navigational hierarchy. Separators are automatically added in CSS through: before and content.

The following code uses the class `.breadcrumb`:

```
<h1>Breadcrumbs</h1>
<ol class="breadcrumb">
  <li><a href="#">Home</a></li>

</ol>
<ol class="breadcrumb">
  <li><a href="#">Home</a></li>
  <li><a href="#">Library</a></li>
  <li class="active">Data</li>
</ol>
<ul class="breadcrumb">
  <li><a href="#">Home</a> <span class="divider"></span></li>
</ul>
<ul class="breadcrumb">
  <li><a href="#">Home</a> <span class="divider"></span></li>
  <li><a href="#">2012</a> <span class="divider"></span></li>
  <li><a href="#">December</a> <span class="divider"></span></li>
```

```
<li><a href="#">5</a></li>
</ul>
```

Home

Home / Library / Data

Home /

Home / / 2012 / / December / / 5

Figure 3-28. Breadcrumb

☺ Pagination

Bootstrap handles pagination like a lot of other interface elements, an unordered list, with wrapper a `<div>` that has a specific class that identifies the element. In the basic form, adding `.pagination` to the parent `<div>` creates a row of bordered links. Each of the list items can be additionally styled by using the `.disabled` or `.active` class. See Figures 3-29 and 3-30 for examples of this.

Default pagination or basic pagination

Simple pagination inspired by Rdio, great for apps and search results. The large block is hard to miss, easily scalable, and provides large click areas.

Here's the code for basic pagination:

```
<div>
  <ul class="pagination">
    <li>
      <a href="#">&laquo;</a></li>
    <li><a href="#">1</a></li>
    <li><a href="#">2</a></li>
    <li><a href="#">3</a></li>
    <li><a href="#">4</a></li>
    <li><a href="#">5</a></li>
    <li>
      <a href="#">&raquo;</a></li>
```

```
</ul>
</div>
```



Figure 3-29. Basic pagination

Disabled and active states

And heres the code for pagination using helper classes:

```
<div>
    <ul class="pagination pagination-centered">
        <li class="disabled"><a href="#"><</a></li>
        <li class="active"><a href="#">1</a></li>
        <li><a href="#">2</a></li>
        <li><a href="#">3</a></li>
        <li><a href="#">4</a></li>
        <li><a href="#">5</a></li>
        <li><a href="#">></a></li>
    </ul>
</div>
```

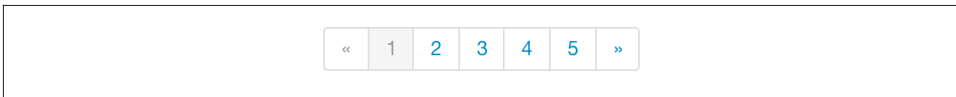


Figure 3-30. Pagination with helper classes

Sizing

In addition to the .active and .disabled classes for list items, you can add .pagination-centered to the parent <div>. This will center the contents of the <div>. If you want the items right-aligned in the <div>, add .pagination-right. For sizing, in addition to the normal size, there are three other sizes that can be applied by adding a class to the wrapper <div>: .pagination-lg, .pagination-sm:

Fancy larger or smaller pagination? Add .pagination-lg or .pagination-sm for additional sizes

```
<nav><ul class="pagination pagination-lg">...</ul></nav>
```

```
<nav><ul class="pagination">...</ul></nav>
```

```
<nav><ul class="pagination pagination-sm">...</ul></nav>
```

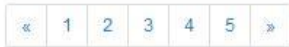
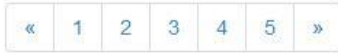
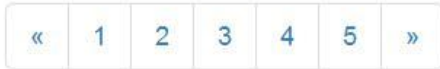


Figure 3-31. Pagination sizes

Pager

If you need to create simple pagination links that go beyond text, the pager can work quite well. Like the pagination links, the markup is an unordered list that sheds the wrapper `<div>`. By default, the links are centered (see [Figure 3-32](#)).

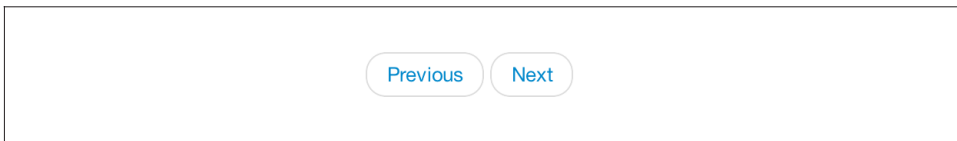


Figure 3-32. Basic pager

The following is the code for a basic pager:

```
<ul class="pager">
  <li><a href="#">Previous</a></li>
  <li><a href="#">Next</a></li>
</ul>
```

To left- or right-align the links, you just need to add the `.previous` and `.next` class as to the list items (see [Figure 3-33](#)). Also, like `.pagination` in [Figure 3-31](#), you can add the `.disabled` class for a muted look.

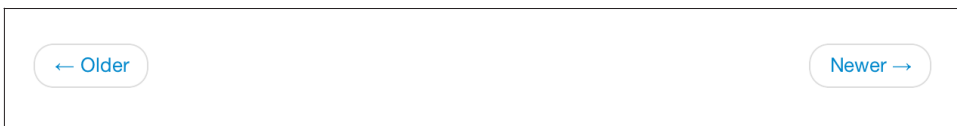


Figure 3-33. Aligned page links

The following is the code for aligning page links:

```

<ul class="pager">
  <li class="previous">
    <a href="#">&larr; Older</a>
  </li>
  <li class="next">
    <a href="#">Newer &rarr;</a>
  </li>
</ul>

```

☺ Labels

Labels are great for offering counts, tips, or other markup for pages. They're another of my favorite little Bootstrap touches. **Figure 3-34** shows some labels that can be used.



Figure 3-34. Labels

Here's the code to use these labels:

```

<span class="label label-default">Default</span>
<span class="label label-primary">Primary</span>
<span class="label label-success">Success</span>
<span class="label label-info">Info</span> <span
class="label label-warning">Warning</span> <span
class="label label-danger">Danger</span>

```

Another example:



```

<h3>Example heading <span
class="label label-

```



```
default">New</span></h3>
```

😊 Badges

Badges are similar to labels; the primary difference is that the corners are more rounded. The colors of badges reflect the same classes as labels (see [Figure 3-35](#)).

badge

1 2 4 6

Figure 3-35. Badges

The following code shows how to use badges:

```
<span class="badge">1</span>
<span class="badge">2</span>
<span class="badge">4</span>
<span class="badge">6</span>
```

Inbox 42

Messages 4

```
<a href="#">Inbox <span class="badge">42</span></a>
```

```
<button class="btn btn-primary" type="button">
  Messages <span class="badge">4</span>
```

```
</button>
```

☺ Typographic Elements

In addition to buttons, labels, forms, tables, and tabs, Bootstrap has a few more elements for basic pagelayout.

Hero Unit

The hero unit is a large content area that increases the size of headings and adds a lot of margin for landing page content (see **Figure 3-36**). To use the hero unit, simply create a container `<div>` with the class of `.hero-unit`. In addition to a larger `<h1>`, the font-weight is reduced to 200 :

```
<div class="hero-unit">
  <h1>Hello, World!</h1>
  <p>This is a simple hero unit, a simple jumbotron-style component for calling extra attention to
  featured content or information.</p>
  <p><a class="btn btn-primary btn-large">Learn more</a></p>
</div>
```

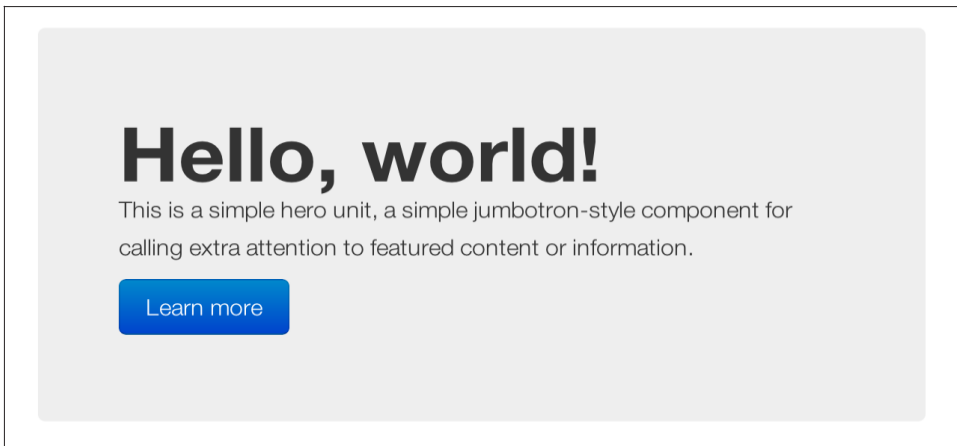


Figure 3-36. Hero unit

Page Header

The page header (see **Figure 3-37**) is a nice little feature to add appropriate spacing around the headings on a page. This is particularly helpful on a blog archive page where you may have several post titles and need a way to add distinction to each of them. To use a page header, wrap your heading in a `<div>` with a class of `.page-header`:

```
<div class="page-header">
  <h1>Example page header <small>Subtext for header</small></h1>
</div>
```

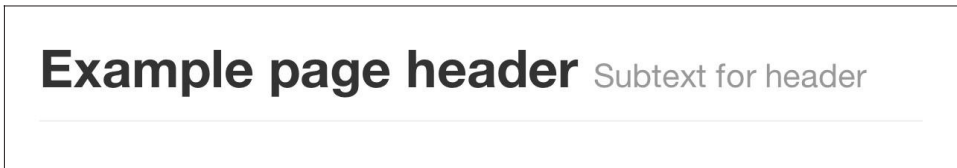


Figure 3-37. Page header

☺ Thumbnails

A lot of sites need a way to lay out images in a grid, and Bootstrap has an easy way to do this. To create a thumbnail, add an `<a>` tag with the class of `.thumbnail` around an image. This adds four pixels of padding and a gray border (see [Figure 3-38](#)). On hover, an animated glow outlines the image.

Default Example

By default, Bootstrap's thumbnails are designed to showcase linked images with minimal required markup.



```
<div class="row">
<div class="col-xs-6 col-md-3">
  <a href="#" class="thumbnail">
     </a>
  </div>
</div>
```

Now that you have your basic thumbnail, you can add headings, buttons, and more as shown in [Figure 3-39](#); just change the `<a>` tag that has a class of `.thumbnail` to a

<div>. Inside of that <div>, you can add anything you need. Since this is a <div>, we can use the default span-based naming convention for sizing. If you want to group multiple images, place them in an unordered list, and each list item will be floated to the left.

CUSTOM CONTENT

With a bit of extra markup, it's possible to add any kind of HTML content like headings, paragraphs, or buttons into thumbnails.



```
<div class="row">
  <div class="col-sm-6 col-md-4">
    <div class="thumbnail">
      
      <div class="caption">
        <h3>Thumbnail label</h3>
        <p>...</p>
        <p><a href="#" class="btn btn-primary"
role="button">Button</a> <a href="#" class="btn btn-default"
role="button">Button</a></p>
      </div>
    </div>
  </div>
</div>
```

☺ Alerts:

Provide contextual feedback messages for typical user actions with the handful of available and flexible alert messages.

Examples

Wrap any text and an optional dismiss button in `.alert` and one of the four contextual classes (e.g., `.alert-success`) for basic alert messages.

No default class

Alerts don't have default classes, only base and modifier classes. A default gray alert doesn't make too much sense, so you're required to specify a type via contextual class. Choose from success, info, warning, or danger.

Well done! You successfully read this important alert message.

Heads up! This alert needs your attention, but it's not super important.

Warning! Better check yourself, you're not looking too good.

Oh snap! Change a few things up and try submitting again.

```
<div class="alert alert-success" role="alert">...</div>
<div class="alert alert-info" role="alert">...</div>
<div class="alert alert-warning" role="alert">...</div>
<div class="alert alert-danger" role="alert">...</div>
```

DISMISSIBLE ALERTS

Build on any alert by adding an optional `.alert-dismissible` and close button.

Requires JavaScript alert plugin

For fully functioning, dismissible alerts, you must use the [alerts JavaScript plugin](#).

Warning! Better check yourself, you're not looking too good.



```
<div class="alert alert-warning alert-dismissible" role="alert">
```

```
    <button type="button" class="close" data-dismiss="alert" aria-label="Close"><span aria-hidden="true">&times;</span></button>
```

```
    <strong>Warning!</strong> Better check yourself, you're not looking too good.
```

```
</div>
```

Ensure proper behavior across all devices

Be sure to use the `<button>` element with the `data-dismiss="alert"` data attribute.

LINKS IN ALERTS

Use the `.alert-link` utility class to quickly provide matching colored links within any alert.

Well done! You successfully read **this important alert message**.

Heads up! This **alert needs your attention**, but it's not super important.

Warning! Better check yourself, you're **not looking too good**.

Oh snap! **Change a few things up** and try submitting again.

```
<div class="alert alert-success" role="alert"> <a href="#" class="alert-link">...</a>
```

```
</div>
```

```
<div class="alert alert-info" role="alert"> <a href="#" class="alert-link">...</a>
```

```
</div>
```

```
<div class="alert alert-warning" role="alert"> <a  
  href="#" class="alert-link">...</a>
```

```
</div>
```

```
<div class="alert alert-danger" role="alert">
```

```
  <a href="#" class="alert-link">...</a>
```

```
</div>
```

😊 Progress Bars

The purpose of progress bars is to show that assets are loading, in progress, or that there is action taking place regarding elements on the page. Personally, I think that these elements are just an exercise in markup and have little purpose beyond that in the

Bootstrap framework. That being said, among the thousands of people using Bootstrap, there are likely a few outliers who have a good reason for building progress bars. By nature, these are static elements that need some sort of JavaScript method to provide any interaction.

The default progress bar has a light gray background and a blue progress bar as shown in **Figure 3-43**. To create it, add a `<div>` with a class of `.progress`. Inside, add an empty

`<div>` with a class of `.bar`. Add a style attribute with the width expressed as a percentage. I added `style="60%";` to indicate that the progress bar was at 60%:

Basic Example

Default Progress Bar.

```
<div class="progress">  
  <div class="progress-bar" style="width: 60%;"></div>  
</div>
```



*Figure 3-43. Default progress bar***With Label**

Remove the `<span>` with `.sr-only` class from within the progress bar to show a visible percentage.



```
<div class="progress">
  <div class="progress-bar" role="progressbar" aria-valuenow="60" aria-
    valuelmin="0" aria-valuemax="100" style="width: 60%;">
    60%
  </div>
</div>
```

To ensure that the label text remains legible even for low percentages, consider adding a `min-width` to the progress bar.

Contextual Alternatives

Progress bars use some of the same button and alert classes for consistent styles.



```
<div class="progress">

  <div class="progress-bar progress-bar-success" role="progressbar" aria-valuenow="40"
    aria-valuemin="0" aria-valuemax="100" style="width: 40%">
    <span class="sr-only">40% Complete (success)</span>

  </div>

</div>

<div class="progress">
  <div class="progress-bar progress-bar-info" role="progressbar" aria-
    valuenow="20" aria-valuemin="0" aria-valuemax="100" style="width: 20%">

    <span class="sr-only">20% Complete</span>
```



```
</div>
```

```
</div>
```

Striped

Uses A Gradient To Create A Striped Effect. Not Available In ie9 And Below.

To create a striped progress bar (see [Figure 3-44](#)),¹ just add `.progress-striped` to the container `<div>`:

```
<div class="progress progress-striped">
  <div class="progress-bar" style="width: 20%;"></div>
</div>
```



Figure 3-44. Striped progress bar

Like the striped version of the progress bar, you can animate the stripes (see [Figure 3-45](#) for a static image of this), making it look like a blue light special barbershop pole.

Animated

Add `.Active` To `.Progress-Bar-Striped` To Animate The Stripes Right To Left. Not Available In ie9 And Below.

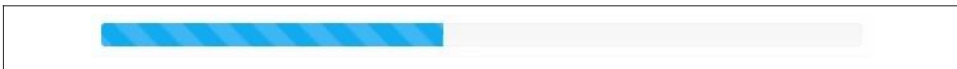


Figure 3-45. Animated progress bar

Here's the code to animate the progress bar:

```
<div class="progress progress-striped active">
  <div class="progress-bar" style="width: 40%;"></div>
</div>
```

Stacked

Place Multiple Bars Into The Same `.Progress` To Stack Them.

In addition to the blue progress bar, there are options for green, yellow, and red using the `.bar-success`, `.bar-warning`, and `.bar-danger` classes. Progress bars can be stacked (see [Figure 3-46](#)), indicating a graph of sorts by adding multiple elements together using this code:



Figure 3-46. Stacked progress bar

```
<div class="progress">

  <div class="progress-bar progress-bar-success" style="width: 35%"> <span class="sr-
    only">35% Complete (success)</span>

  </div>

  <div class="progress-bar progress-bar-warning progress-bar-striped"
style="width: 20%">

    <span class="sr-only">20% Complete (warning)</span> </div>

  <div class="progress-bar progress-bar-danger" style="width: 10%"> <span class="sr-
    only">10% Complete (danger)</span>

  </div>

</div>
```

😊 Wells

A well is a container `<div>` that causes the content to appear sunken on the page (see [Figure 3-50](#)). I have used wells for blog post meta information like author, date, and categories. To create a well, simply wrap the content that you would like to appear in the well with a `<div>` containing the class of `.well`:

Default Well

Use the well as a simple effect on an element to give it an inset effect.

```
<div class="well">
    Look, I'm in a .well!!
</div>
```



Figure 3-50. Well

Optional Classes

Control padding and rounded corners with two optional modifier classes. There are two additional classes that can be used in conjunction with `.well`: `.well-lg` and `.well-sm`. These affect the padding, making the well larger or smaller depending on the class (see [Figure 3-51](#)).



Figure 3-51. Well optional classes

The following code uses the well classes:

```
<div class="well well-lg">
    Look, I'm in a .well-large!
</div>

<div class="well well-sm">
    Look, I'm in a .well-small!
</div>
```